Diffusion Model

Hung Manh

Ngày 21 tháng 10 năm 2026

Nội dung trình bày

1. Looped Flows: Tóm tắt
2. Looped Model: Giới thiệu
3. LM: Kiến thức nền tảng
4. LM: Phương pháp luận
5. LM: Nghiên cứu liên quan
6. LM: Thử nghiệm và Đánh giá
7. LM: Kết luận
8. Shared Memory in Looped Transformers
9. FMLM: Tóm tắt
10. FMLM: Bối cảnh & Thách thức
11. FMLM: Mô hình cơ sở FLM
12. FMLM: Phương pháp luận Flow Map
13. FMLM: Dẫn hướng trong suy luận
14. FMLM: Thử nghiệm và Đánh giá
15. FMLM: Kết luận

Looped Flows: Tóm tắt

- **Vấn đề:** Giải bài toán khó cần nhiều compute lúc inference. Looped model làm điều này bằng cách cập nhật hidden state lặp lại với tham số share. Tuy nhiên, BPTT đầy đủ qua nhiều vòng thì gradient vanishing/exploding, nên thực tế chỉ backprop qua một hoặc vài vòng cuối. Do đó các vòng đầu không nhận được tín hiệu học về việc chúng nên tạo ra state thế nào để hỗ trợ tính toán về sau.
- **Đề xuất: Looped flows** - huấn luyện recurrency với các mục tiêu khử nhiễu cục bộ.
- **Cơ chế:**
  - *Temporal alignment*: Giảm dần mức độ nhiễu và sử dụng shared noise.
  - *Inference*: Tích phân vận tốc của probability flow kết hợp với các looped states.
- **Kết quả:** Vượt qua các SOTA looped models, đạt **58.8%** trên ARC-AGI-1 và **12.2%** trên ARC-AGI-2; cho phép scale tính toán và tạo multiple valid predictions.

1. Giới thiệu: Bối cảnh & Thách thức

Bối cảnh:

- Giải bài toán khó đòi hỏi gia tăng tính toán lúc inference.
- Khác với Autoregressive LLMs, các looped models sử dụng distributed state và cập nhật tuần hoàn để tích lũy computation.

Thách thức của Looped Models:

- Huấn luyện bằng full BPTT tốn kém bộ nhớ và dễ gặp hiện tượng vanishing/exploding gradients.
- Thực tế chỉ backprop qua vài bước $\rightarrow$ early updates không nhận được tín hiệu để hỗ trợ future computation.
- Hậu quả: Chuỗi lặp không ổn định, sinh ra các failure modes ở cả bài toán đơn giản.

1. Giới thiệu: Giải pháp Looped Flows

Đóng góp chính của Looped Flows:

- Học một chuỗi mục tiêu khử nhiễu cục bộ theo thời gian ở nhiều mức nhiễu khác nhau.
- **Temporal alignment:** Phân bổ huấn luyện theo thời gian, giảm dần mức nhiễu, sử dụng shared noise $\rightarrow$ hidden states duy trì tính ổn định.
- **Cơ chế Inference:** Giải bài toán bằng cách tích phân trường vận tốc của probability flow kết hợp với recurrent states.
- **Lợi ích:**
  - Mở rộng tính toán: Chia quá trình suy luận thành nhiều bước thời gian nhỏ hơn để tăng độ chính xác.
  - Tạo nhiều dự đoán hợp lệ: Sử dụng phương pháp tích phân Euler hoặc tích phân ngẫu nhiên để tạo ra nhiều phương án dự đoán.
  - Học vòng lặp ổn định: Giúp mô hình học cách cập nhật trạng thái ổn định hơn, khắc phục những hạn chế của các mô hình lặp trước đây.

2. Kiến thức nền: Looped Models

- **Định nghĩa:** Cập nhật hidden state $z \in \mathbb{R}^{d_z}$ qua hàm f:

$$z_i = f(z_{i-1}; c)$$

- Đưa ra dự đoán qua đầu đọc g: $\hat{x}_i = g(z_i)$.
- **Hàm loss thực tế:** Để tránh full BPTT, mô hình dùng toán tử dừng gradient $sg(\cdot)$:

$$L_{loop}(f, g) := \mathbb{E}_{c,x}\left[\sum_i CE(\hat{x}_i, x)\right] \quad \text{với } z_i = f(sg(z_{i-1}); c)$$

- *Điểm yếu:* Loss tại bước $\geq i$ không truyền tín hiệu về quá khứ $z_{<i}$. Quá trình lặp deterministic chỉ cho phép 1 lời giải duy nhất.

2. Kiến thức nền: Flow and Diffusion Models

- **Continuous flow matching:** Đi từ nhiễu $x_0 \sim \mathcal{N}(0, \sigma^2 I)$ đến dữ liệu thật $x_1 \sim p_{data}$.
- **Probability path:** Nội suy $I_t := (1-t)x_0 + tx_1$.
- **Velocity field:** $\dot{x}_t = b_t(x_t; c)$.
- Tích phân số (VD: Euler tiến) để xấp xỉ lời giải:

$$x_{t_{i+1}} = x_{t_i} + (t_{i+1} - t_i)\hat{b}_{t_i}(x_{t_i}; c)$$

- **Denoising function (D_t):** Thay vì dự đoán trực tiếp vận tốc, học hàm khử nhiễu độc lập tại mỗi t:

$$L_{flow}(\hat{D}) := \mathbb{E}[CE(\hat{x}_t, x_1)]$$

- *Nhược điểm:* Mô hình phụ thuộc hoàn toàn vào quá trình flow để cập nhật trạng thái, nên khó học các phép tính cần thực hiện theo đúng thứ tự qua nhiều bước.

3. Looped Flows: Cơ chế Huấn luyện

Stateful Denoiser:

- Khử nhiễu x_{t_i} đồng thời cập nhật looped state z_{t_i}:

$$(\hat{x}_{t_i}, z_{t_i+1}) = \hat{D}_{t_i}(I_{t_i}, sg(z_{t_i}); c)$$

Temporal Alignment:

- Sắp xếp lưới thời gian $t_0 < t_1 < \cdots < t_k$ để mức nhiễu giảm dần.
- **Shared noise:** Mẫu x_0 dùng chung cho toàn bộ quỹ đạo $\rightarrow$ ép mô hình liên kết các thời điểm, giúp z_{t_i} mang thông tin hữu ích cho tương lai.

Adaptive Computation Time (ACT):

- Dùng đầu đọc phân loại nhị phân $q(z_{t_i+1})$ để quyết định dừng suy luận sớm khi độ chính xác đã bão hòa.

3. Looped Flows: Cơ chế Suy luận

- **Stochastic Integration:**
  - Thêm hệ số ngẫu nhiên $\gamma \geq 0$.
  - Lùi nhẹ x_{t_i} về thời điểm $s < t_i$ (tiêm nhiễu mới) trước khi thực hiện bước Euler tiến.
  - Khi $\gamma = 0$, mô hình quay về Euler ODE truyền thống.
- **Inference-time Scaling:**
  - Giải bài toán bằng finer time grid so với lúc huấn luyện.
  - Sử dụng kỹ thuật **best-Q ensembling**: Chạy nhiều mẫu, chọn dự đoán có xác suất thành công cao nhất từ đầu đọc ACT q.

4. Nghiên cứu liên quan

- **Looped models (TRM, FPRM):** Thiếu giám sát toàn cục do cắt gradient, dễ rơi vào spurious attractors. *Looped flows giải quyết bằng cách khởi tạo lặp từ local denoising.*
- **Stochastic looped models:** Tận dụng nhiễu nhưng vẫn phụ thuộc gradient cục bộ. *Looped flows kết hợp lợi ích ngẫu nhiên và giải quyết triệt để học cục bộ.*
- **Flow & Diffusion Models:** Học khử nhiễu độc lập. *Looped flows dùng local objectives tạo lộ trình học mượt mà cho hidden state.*
- **Energy models:** *Looped flows tương đương việc học chuỗi năng lượng làm mịn giảm dần*, giúp quá trình tối ưu dễ dàng hơn.

5. Thử nghiệm: Chính xác & Khả năng mở rộng

So sánh đơn quỹ đạo (Single-trajectory):

- Looped flows áp đảo TRM, FPRM, GRAM trên:
  - Sudoku-Extreme (**97.9%**)
  - Maze-Hard (**86.7%**)
  - ARC-AGI-1 (**58.8%** - tăng từ 44.6% của TRM)
  - ARC-AGI-2 (**12.2%** - tăng từ 7.8% của TRM)

Inference-time scaling:

- Ensembling cải thiện thêm hiệu năng (Sudoku 99.3%, ARC 59.5%).
- Finer time grid tăng accuracy mạnh: Sudoku từ 74.5% (8 steps) $\rightarrow$ 97.9% (128 steps).

Độ ổn định:

- Giải quyết 90.9% các trường hợp thất bại của TRM (bao gồm non-convergence và spurious attractors).

5. Thử nghiệm:

Diversity: Đánh giá trên bài toán đa nghiệm.

- N-Queens (8x8): **99.9%** ACC / **91.4%** Coverage.
- Graph Coloring (8 đỉnh): **0.7** conflicts / **89.4%** Coverage.
- Vượt trội hoàn toàn về khả năng tìm ra tập lời giải đa dạng nhờ probability transport.

Ablation Study:

- **Flow:** Bỏ time conditioning, bỏ interpolant, không giảm nhiễu, hay không chia sẻ nhiễu đều làm giảm mạnh hiệu năng.
- **Recurrence:** So với mô hình không lặp (FLM) hoặc self-conditioning, Looped flows đạt khả năng tổng quát hóa tốt nhất trên test set.
- Tích phân SDE ($\gamma = 5$) cho kết quả tốt hơn Euler ODE ($\gamma = 0$).

6. Kết luận

- **Looped Flows** là một framework giải quyết vấn đề huấn luyện của Looped Models thông qua các mục tiêu khử nhiễu cục bộ.
- **Temporal alignment** giúp trạng thái lặp xây dựng computation hữu ích trên toàn cục, khắc phục hoàn toàn điểm yếu của gradient ngắn hạn.
- Mở ra khả năng **inference-time scaling** mạnh mẽ qua lưới thời gian mịn và tích phân ngẫu nhiên.
- **Hướng tương lai:** Phát triển các thuật toán huấn luyện không cần mô phỏng (simulation-free training).

LPT: Tổng quan bài báo

- **Tên bài báo:** *The Surprising Effectiveness of Shared Memory in Looped Transformers*
- **Tác giả & Tổ chức:** Giovanni Monea, Keshav Ramji, Yousef El-Kurdi et al. (Cornell University & IBM Research, arXiv:2610.02383)
- **Ý tưởng cốt lõi:** Huấn luyện Looped Transformers chia sẻ bộ nhớ ngay từ đầu (pre-trained shared memory) thay vì lưu cache riêng cho từng bước lặp.
- **Đóng góp nổi bật:**
  - **Xóa bỏ định kiến:** Cắt giảm bộ nhớ không làm giảm chất lượng mà ngược lại giúp tăng mạnh độ chính xác và giảm Perplexity.
  - **Tiết kiệm bộ nhớ vượt bậc:** Giảm **76–79%** context memory so với Standard Transformer cùng kích thước (150M $\sim$ 1B).
  - **Đại lộ Gradient (Gradient Highway):** Cơ chế đọc shared memory giúp dòng gradient từ hàm loss bước cuối truyền thẳng về vòng lặp đầu tiên, ổn định đệ quy sâu.

1. LPT: Bối cảnh & bộ nhớ KV Cache

Nguyên lý Looped Transformers :

- Áp dụng cùng một khối L tầng Transformer lặp lại R lần trên hidden state trước khi dự đoán token:
$$h_{\text{in}}^{(t+1)} = \mathcal{N}_{\text{rec}}\left(h^{(t)} + \mathcal{M}^{L}(h_{\text{in}}^{(t)})\right)$$
- Đạt độ sâu tính toán hiệu dụng $L \cdot R$ nhưng chỉ tốn số lượng tham số của L tầng.

Context Memory Explosion:

- Trong LT truyền thống, mỗi recursion $t \in [1, R]$ ghi một bảng KV cache riêng biệt.
- Tổng dung lượng cache: $\mathcal{O}(R \cdot L \cdot T \cdot d)$. Tại $T = 4096$, bộ nhớ KV cache phình to vượt xa bộ nhớ lưu trữ trọng số mô hình.
- *Thất bại của giải pháp cũ:* Bỏ bớt cache lúc suy luận (post-hoc inference drop như Ouro, Huginn) khiến chất lượng sinh sụp đổ nghiêm trọng.

2. LPT: Kiến trúc Looped Prediction Transformer

Phân tách Bộ nhớ và Dự đoán theo trục đệ quy:

- **State Recursion ($t = 1$):** Đọc toàn bộ ngữ cảnh, xây dựng và ghi lại bảng full-context KV cache duy nhất $\mathcal{C}^{(1)}$ trên toàn bộ L tầng.
- **Prediction Recursions ($t \geq 2$):** Không ghi thêm full cache, chỉ duy trì một cửa sổ trượt cục bộ siêu ngắn $\omega = 64$ token ($\mathcal{C}^{(t)}_{[i-\omega,i]}$, chiếm $< 2\%$ ngữ cảnh).

Attention đồng nhất qua một phép Softmax duy nhất:

$$\mathcal{A}_i^{(t)} = \mathcal{C}^{(1)}_{1:i} \cup \mathcal{C}^{(t)}_{[i-\omega,i]}$$

- Query $q_i^{(t)}$ chú ý đồng thời vào Shared Memory và cửa sổ cục bộ trong một lượt tính toán (tối ưu hóa qua FlexAttention block mask).
- **Biến thể Hybrid (H-LPT):** Kết hợp các tầng Linear Attention (Gated DeltaNet) với các tầng Full Attention dùng Shared Memory theo tỷ lệ 4:1.

3. LPT: Kết quả thực nghiệm phá vỡ biên giới Quality–Memory

Đánh giá trên FineWeb-Edu (Context $T = 4096$, Quy mô 150M – 1B tham số):

Mô hình	Recursions (R)	Val PPL (↓)	Zero-shot Acc (↑)	C
Standard Transformer (STD)	1	Baseline	Baseline	
Vanilla LT (Full Cache)	5	Dừng cải thiện	Thấp hơn LPT 0.4 $\sim$ 4.8%	
LPT (Ours)	5	**-1.09 nats** vs LT	**+4.8%** vs LT	
H-LPT (Ours)	5	**-1.82 nats** vs STD	**+5.6%** vs STD	**G**

- **Quy luật mở rộng theo độ sâu:** LT truyền thống dừng cải thiện sau 3 vòng lặp; LPT tiếp tục tăng trưởng chất lượng mượt mà đến 5 vòng lặp.
- **Huấn luyện đơn giản:** LPT chỉ cần dùng Final Loss đơn giản ở bước cuối, không cần cơ chế Adaptive loss phức tạp như Ouro mà vẫn cực kỳ ổn định.

4. LPT: Gradient Highway

Nghịch lý suy giảm gradient trong đệ quy sâu:

- Trong LT chuẩn, gradient từ hàm loss ở bước cuối bị suy giảm nghiêm trọng khi truyền ngược qua nhiều bước lặp tuần tự $\rightarrow$ vòng 1 không nhận đủ tín hiệu.

Shared Memory đóng vai trò Gradient Highway:

- Do query ở các recursion sau đọc trực tiếp Shared KV cache của vòng 1, gradient có một đường dẫn tắt truyền thẳng về vòng 1.
- Chuẩn gradient truyền về vòng 1 trong LPT được phục hồi tương đương Standard Transformer, không bị sụt giảm theo số vòng lặp.

Thí nghiệm triệt tiêu (Ablation):

- Khi ngắt dòng gradient này (*detach shared KV*), mô hình ngay lập tức bị suy giảm chất lượng nặng nề hoặc không thể hội tụ khi tăng số vòng lặp.

5. LPT: Phân hóa biểu diễn & Vai trò của ω

Phân hóa không gian biểu diễn (PCA không gian Key):

- Phân tích PCA cho thấy Shared Keys và Local Keys tự động phân tách thành hai cụm không gian hoàn toàn riêng biệt.
- *Shared Keys:* Chuyên biệt hóa lưu trữ ngữ cảnh tĩnh toàn cục.
- *Local Keys:* Chuyên biệt hóa xử lý các tương tác cục bộ ngắn hạn.

Thí nghiệm quét kích thước cửa sổ ω ($0 \to 4096$):

- LPT vượt trội LT chuẩn ở **mọi kích thước cửa sổ** ω.
- Ngay cả khi $\omega = 0$ (hoàn toàn không lưu cache quá khứ ở các vòng sau), LPT vẫn tốt hơn LT giữ full cache.
- Tỷ trọng Attention: Càng về các recursion sau, mô hình càng dồn sự chú ý vào Shared Memory (chiếm tới **73–80%** attention mass).

6. LPT: Đo đạc thực tế trên phần cứng NVIDIA H100

Khắc phục nút thắt Memory-Bound khi phục vụ mô hình:

- **Khả năng chứa Batch (Batch Capacity):** Nhờ cắt giảm 79% context memory, H-LPT cho phép tăng batch size lên gấp $\mathbf{3.6\times - 5.0\times}$ so với Standard Transformer trước khi đầy 80GB VRAM.
- **Thông lượng thực tế (Throughput):**
  - Ở $R = 3$: H-LPT đạt thông lượng cao gấp $\mathbf{1.4 - 1.5\times}$ so với Standard Transformer.
  - Ở $R = 5$: H-LPT duy trì thông lượng **81–88%** so với Standard Transformer, dù thực hiện số phép tính gấp nhiều lần.

Ý nghĩa thực tiễn

Việc tăng batch size trên GPU bù đắp hoàn toàn chi phí tính toán lặp lại, biến Looped Transformers thành giải pháp khả thi trong môi trường sản xuất thực tế.

7. LPT: Kết luận & Định hướng tương lai

- **Inductive Bias mới:** Chia sẻ bộ nhớ không phải là sự đánh đổi chấp nhận mất mát, mà là một cấu trúc kiến trúc vượt trội giúp phân định rõ ràng giữa lưu trữ ngữ cảnh và tinh chỉnh suy luận.
- **Đột phá độ quy sâu:** Gradient Highway qua shared cache mở đường cho việc mở rộng Looped Transformers lên số vòng lặp lớn mà không gặp hiện tượng vanishing gradient.
- **Hiệu quả triển khai:** Giảm 76–79% bộ nhớ ngữ cảnh, tăng 5$\times$ batch capacity trên GPU hiện đại.

Hướng phát triển tiếp theo:

- Tích hợp cơ chế dừng sớm linh hoạt (adaptive compute / early exit) theo từng độ khó của token.
- Mở rộng sang các bài toán suy luận đa bước phức tạp (complex multi-step reasoning).

FMLM: Tóm tắt bài báo

- **Vấn đề:** Discrete diffusion models sinh song song thay thế autoregressive LLMs, nhưng chất lượng ko tốt ở few-step regime ($1 \sim 4$ bước) do lỗi factorization approximation.
- **Đề xuất: Flow Map Language Models (FMLM)** — mô hình hóa ngôn ngữ bằng continuous flows trên biểu diễn one-hot unconstrained.
- **Cơ chế:**
  - *Flow Language Model (FLM)*: Đưa bài toán về denoiser trên simplex, huấn luyện bằng cross-entropy và time reparameterization theo decoding error.
  - *Two-time denoiser ($\delta_{s,t}$)*: Tham số hóa flow map nằm trọn trên simplex, chưng cất bằng semigroup condition.
- **Kết quả:** FMLM đạt khả năng sinh văn bản one-step vượt qua chất lượng 8 bước của discrete diffusion baselines, tăng tốc **8.3**$\times$, hỗ trợ continuous guidance ổn định.

FMLM: Giới hạn của Discrete Diffusion

Hạn chế của Autoregressive LLMs:

- Sinh tuần tự từng token một ($O(L)$ bước tính toán), tạo ra điểm nghẽn lớn về tốc độ khi sinh chuỗi dài.

Factorization Error của Discrete Diffusion:

- Discrete diffusion (MDLM, Duo) khử nhiễu trên không gian chuỗi $\mathcal{V}^L$.
- Do không gian trạng thái quá lớn, mô hình bắt buộc phải dùng xấp xỉ phân rã tích:

$$\tilde{p}_{t|s}(\mathrm{y}_t|\mathrm{y}_s) := \prod_{l=1}^{L} p^l_{t|s}(y^l_t|\mathrm{y}_s)$$

- Giả định các token độc lập có điều kiện chỉ đúng trong giới hạn vô cùng bé ($t \to s$).
- *Hậu quả ở few-step ($1 \sim 4$ bước):* Bước nhảy lớn làm mất hoàn toàn tương quan ngữ nghĩa giữa các token $\to$ văn bản vô nghĩa (PPL > 1000) hoặc lặp từ liên tục (entropy collapse < 4.0).

FMLM: Paradigm Shift

Bác bỏ định kiến về Discrete Modalities:

- Quan niệm phổ biến cho rằng dữ liệu rời rạc bắt buộc phải dùng discrete noising process.
- **Giới hạn toán học:** Trong không gian rời rạc hữu hạn S, không tồn tại bất kỳ deterministic map $f : S \to S$ nào trên từng mẫu để biến đổi một phân phối tùy ý về phân phối đích.
- Hệ quả: discrete diffusion không thể có flow map tất định trên từng mẫu để đưa nhiễu về dữ liệu chỉ trong 1 bước.

Giải pháp Continuous Flows:

- Nhúng token dạng one-hot lossless vào không gian Euclid không ràng buộc $\mathbb{R}^{L \times |\mathcal{V}|}$.
- Dòng liên tục trong $\mathbb{R}^{L \times |\mathcal{V}|}$ sở hữu một flow map tất định duy nhất trên từng mẫu, mở đường cho việc sinh 1 bước chính xác.

FMLM: Mô hình cơ sở Flow LM (FLM)

- **Stochastic Interpolant:** Nội suy đường đi giữa Gaussian noise $x_0 \sim \mathcal{N}(0, I)$ và one-hot data x_1:

$$I_t := (1 - t)x_0 + tx_1$$

- **Thách thức dự đoán velocity:** Học vận tốc $b_t = \mathbb{E}[x_1 - x_0 | I_t]$ bằng MSE bị rank bottleneck do x_0 full-rank $\rightarrow$ underfitting.
- **Chuyển sang Denoiser trên Simplex:**

$$D_t(x) := \mathbb{E}[x_1 | I_t = x], \quad b_t(x) = \frac{D_t(x) - x}{1 - t}$$

  - $D_t(x)^l = p^l_{1|t}(\cdot | I_t = x) \in \Delta^{|\mathcal{V}|-1}$ (luôn nằm trên simplex xác suất).
  - Huấn luyện bằng **Cross-Entropy loss** thích ứng với hình học one-hot, chặn khoảng cách Wasserstein $W_2^2 \leq C\epsilon$.

FMLM: Time Reparameterization

- **Vấn đề:** Tỷ lệ lỗi giải mã $P_e(t) := \frac{1}{L}\sum_l \mathbb{P}(g^l(\mathrm{x}_t) \neq g^l(\mathrm{x}_1))$ dốc đứng sát $t = 1$.
  - Lấy mẫu đều $t \sim \mathcal{U}[0,1]$ lãng phí tín hiệu ở vùng chưa quyết định token và bỏ sót vùng quyết định then chốt khi $|\mathcal{V}|$ lớn.
- **Đề xuất hàm $\tau(t)$ chuẩn hóa tiến độ giải mã:**

$$\tau(t) = 1 - \frac{|\mathcal{V}|}{|\mathcal{V}| - 1} P_e(t)$$

- **Lợi ích:**
  - Huấn luyện và lấy mẫu đều trong không gian $\tau(t) \sim \mathcal{U}[0,1]$.
  - Mỗi bước tính toán đóng góp đồng đều vào việc giảm decoding error.
  - Giúp mô hình huấn luyện ổn định với từ vựng lớn $|\mathcal{V}| \approx 50.000$.

FMLM: Two-time Denoiser trên Simplex

Mục tiêu Flow Map:

- Ánh xạ trực tiếp giữa hai thời điểm bất kỳ: $X_{s,t}(\mathrm{x}) = \mathrm{x} + (t-s)\mathrm{v}_{s,t}(\mathrm{x})$.
- Cho phép sinh 1 bước qua $\hat{\mathrm{x}}_1 = X_{0,1}(\mathrm{x}_0)$.
- *Thách thức:* $X_{s,t}$ và $\mathrm{v}_{s,t}$ rời khỏi simplex do nhiễu Gaussian $\rightarrow$ dùng MSE cho chất lượng rất kém.

Định nghĩa Two-time Denoiser ($\delta_{s,t}$):

$$\delta_{s,t}(\mathrm{x}) := \mathrm{x} + (1-s)\mathrm{v}_{s,t}(\mathrm{x})$$

4 Tính chất cốt lõi:

1. Phục hồi chính xác Flow Map: $X_{s,t}(\mathrm{x}) = \frac{1-t}{1-s}\mathrm{x} + \frac{t-s}{1-s}\delta_{s,t}(\mathrm{x})$.
2. Bảo toàn trên simplex: $\delta_{s,t}(\mathrm{x})^l \in \Delta^{|\mathcal{V}|-1} \rightarrow$ dùng được softmax output.
3. Khớp với denoiser thông thường trên đường chéo: $\delta_{s,s}(\mathrm{x}) = D_s(\mathrm{x})$.
4. Thỏa mãn semigroup dạng tổ hợp lồi: $\delta_{s,t} = \gamma\delta_{s,u} + (1-\gamma)\delta_{u,t}(X_{s,u})$.

Hàm mất mát Semigroup Distillation:

- Tận dụng tính chất tổ hợp lồi trên simplex, tối ưu qua KL divergence:

$$\mathcal{L}_{\mathrm{KL}}(\hat{\delta}) = \mathbb{E}_{s,u,t}\left[\sum_{l=1}^{L} \mathrm{KL}(\bar{\delta}^{l}_{s,t} \parallel \hat{\delta}^{l}_{s,t}(I_s))\right] + \mathbb{E}_{t}\left[\sum_{l=1}^{L} \mathrm{KL}(\hat{D}_t(I_t)^{l} \parallel \hat{\delta}^{l}_{t,t}(I_t))\right]$$

với teacher: $\bar{\delta}_{s,t} := sg\left(\gamma\hat{\delta}_{s,u}(I_s) + (1-\gamma)\hat{\delta}_{u,t}(\hat{X}_{s,u})\right)$, và $\hat{D}_t$ là FLM teacher đã đóng băng.

Kỹ thuật then chốt:

- **Boundary sampling:** Do $\tau(t)$ phẳng gần $t = 0$, cố định xác suất $p = 1/32$ lấy mẫu cặp $(s, t) = (0, 1)$ để bơm gradient mạnh cho bước nhảy 1-step.
- **One-step inference:** $\hat{x}_1 = \hat{\delta}_{0,1}(x_0)$ sau đó decode qua argmax.

FMLM: Continuous Inference Guidance

Autoguidance (Nâng cao chất lượng sinh không điều kiện):

- Ngoại suy vận tốc ra xa model yếu: $\hat{b}_t^{(\text{guided})} = \hat{b}_t^{(\text{weak})} + \eta(\hat{b}_t - \hat{b}_t^{(\text{weak})})$.
- Nhờ hoạt động trong không gian Euclid, FLM duy trì tính ổn định ở η lên đến **100** (giảm PPL từ 96.9 xuống **51.6** tại $\eta = 50$).
- Discrete diffusion kq ko tốt ở $\eta \geq 10$ do ngoại suy logit nhân bản factorization error.

Reward Guidance (FMRG - Điều hướng theo thuộc tính):

- Flow map $X_{t_n,1}$ cung cấp khả năng **look-ahead vi phân** tới điểm cuối:

$$x_{t_{n+1}} = X_{t_n,t_{n+1}}(x_{t_n}) + \lambda \nabla r\left(X_{t_n,1}(x_{t_n})\right)$$

- Classifier/Reward model r chỉ cần huấn luyện trên **văn bản sạch**, không cần train trên dữ liệu nhiễu như discrete diffusion.

FMLM: Hiệu năng sinh nhiều bước & Đột phá One-step

Chế độ nhiều bước (1024 steps):

- FLM vượt trội các baseline SOTA trên cả LM1B và OpenWebText (OWT):
  - LM1B ($L = 128$): FLM PPL **96.91** vs Duo 98.14, MDLM 109.21, CANDI 120.99.
  - OWT ($L = 1024$): FLM PPL **62.23** vs Duo 77.69, MDLM 105.15.

Chế độ cực ít bước (1, 2, 4 steps) — FMLM One-step Breakthrough:

- Discrete diffusion chưng cất (MDLM+SDTT, Duo+DCD, Di4C) sụp đổ ở 1 bước (PPL > 1000 hoặc entropy collapse < 4.0).
- **FMLM 1-step:** Đạt PPL **119.34** trên LM1B và **168.30** trên OWT.
  - Chất lượng 1 bước tương đương mô hình rời rạc chưng cất ở $8 \sim 16$ bước.
  - Tăng tốc độ sinh **8.3**$\times$.
  - Self-BLEU (0.073) và GPT-4.1 Judge xác nhận không bị mode collapse.

FMLM: Cấu trúc logic (Sudoku) & Ablation Study

Mô hình hóa ràng buộc logic toàn cục (Sudoku 9×9):

- *Unconditional 1-step:* FMLM đạt **9.38%** lời giải hợp lệ (discrete baselines đều $= 0.00\%$).
- *Conditional 1-step (20 clues):* FMLM giải đúng **39.84%** chỉ trong 1 forward pass.

Ablation Study (Bảng 18):

- **Loss & Parameterization:** Denoiser + Softmax + CE (**96.91**) $\gg$ Denoiser + MSE (129.04) $\gg$ Velocity + MSE (3801.36 - không hội tụ).
- **Time Schedule:** Decoding error schedule $\tau(t)$ (**106.98**) $\gg$ Uniform (149.18).
- **Distillation Objective:** Semigroup (**119.34**) $\gg$ Eulerian (149.13) $\gg$ Lagrangian (193.08) do không cần JVP tốn kém và bất ổn định.
- **Boundary sampling ($p = 1/32$):** Cải thiện PPL 1-step từ 142.61 xuống **119.34**.

FMLM: Kết luận & Định hướng

- **Continuous Flow vượt trội trên ngôn ngữ:** Bác bỏ giả thuyết bắt buộc phải dùng discrete diffusion cho dữ liệu rời rạc; continuous flow cho chất lượng cao hơn và suy luận nhanh hơn.
- **Two-time Denoiser trên Simplex:** Cầu nối lý thuyết giúp chưng cất flow map ổn định bằng cross-entropy / semigroup condition.
- **Khả năng mở rộng:** Hỗ trợ autoguidance ổn định và reward-guided generation hiệu quả với look-ahead vi phân.
- **Hạn chế & Hướng tương lai:**
  - Ma trận nhúng $O(L|\mathcal{V}|)$ tốn thêm khoảng 30% bộ nhớ lúc huấn luyện $\rightarrow$ hướng giải quyết bằng sparse gradients.
  - Tiềm năng ứng dụng lớn cho reinforcement learning finetuning (RLFT) nhờ giảm chi phí rollout tính reward.

Cảm ơn bạn đã theo dõi!

Q&A